

线段树

线段树可以用来求解大部分区间问题，他的用途很广泛，今天主要介绍一种最长串的问题。

有一个 n 个元素的数组，给出每个元素初始状态（不是0就是1）。有 m 条指令，指令有3种：

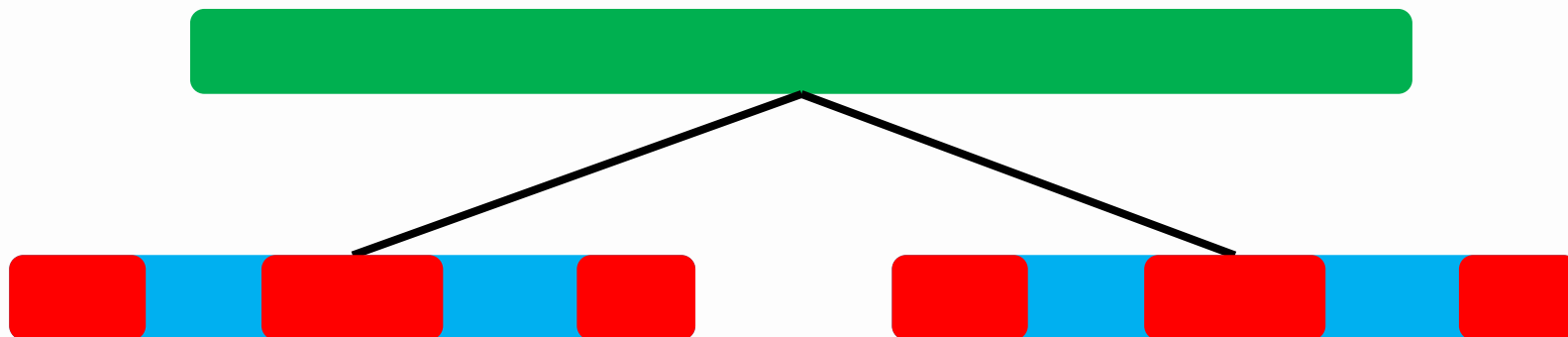
0. 将 $l \cdots r$ 的所有元素改为0
1. 将 $l \cdots r$ 的所有元素改为1
2. 统计 $l \cdots$ 中最长的连续1的长度

建树

(1) 建树

建树很容易，和普通线段树建树一样，当区间长度为1的时候，如果值为1，更新长度为1，值为0，更新长度为0。

但是，当求解父亲结点的区间长度的时候就需要变化了。观察下图。已知两个孩子结点的最长1的长度，其父亲结点的最长1的长度可以直接取max得到吗？



红色表示1，蓝色表示0

建树

并不是，可能就是其左右子区间的最长长度的最大值，但是也有可能不是，因为此时左右两个子区间构成了一个大区间，很有可能左区间的右边+右区间的左边组成的连续的1的数量更大，所以我们不能仅仅只维护区间最长1的长度，还需要维护区间左右两端最长1的长度。

```
struct node
{
    int l, r, lazy, len, lmax, rmax, zmax;
} tree[maxn*4];
```



红色表示1，蓝色表示0

建树

```
void updatafa(int id)
{
    if(tree[id*2].lmax==tree[id*2].len)//左子树全1
        tree[id].lmax=tree[id*2].len+tree[id*2+1].lmax;
    else
        tree[id].lmax=tree[id*2].lmax;
    if(tree[id*2+1].rmax==tree[id*2+1].len)//右子树全1
        tree[id].rmax=tree[id*2+1].len+tree[id*2].rmax;
    else
        tree[id].rmax=tree[id*2+1].rmax;
    tree[id].zmax=max(tree[id*2].zmax,tree[id*2+1].zmax);
    tree[id].zmax=max(tree[id].zmax,tree[id*2].rmax+tree[id*2+1].lmax);
}
```

建树

我们再来考虑父亲区间的左右总三个的长度。
我们会发现左右连续1的长度也有两种情况



父亲区间的左最长=左孩子区间的左最长



父亲区间的左最长=左孩子区间长度+右孩子区间左最长

父亲区间的总最长= $\max$ (左孩子区间总最长, 右孩子区间总最长, 左孩子区间右最长+右孩子区间左最长)

区间更新

更新也是在原来的线段树更新上修改：

如果区间变为0，那么这一段的所有max的值都变为0

如果区间变为1，那么这一段的所有max的值都变为len

如果需要访问到下层区间，判断是否有懒标记，如果有，先把懒标记下传。

子区间更新，其父亲区间也需要更新，更新方式和建树时的更新一模一样。所以可以把更新父亲区间封装成一个函数，这里直接调用就可以了

区间查询

区间查询就要稍微复杂一点，因为查询的区间在线段树上可能由好几部分组成，最终的总的最长可能和前面建树时讨论的情况一样，可能是组合的，所以返回值的时候要注意，不能直接返回最长的长度，而是应返回一个结构体，因为我们还需要当前区间的左右总最长。

查询结束之后，更新父亲区间，更新方式和前面建树时的更新比较类似，分别更新这个结点的左右总最长，并且把这个结构体返回回去。

区间查询

```
if(r<=mid)
return query(id*2,l,r);
else if(l>mid)
return query(id*2+1,l,r);
else
{
    node t1=query(id*2,l,mid);
    node t2=query(id*2+1,mid+1,r);
    node tmp;
    if(t1.lmax==t1.len)
    tmp.lmax=t1.len+t2.lmax;
    else
    tmp.lmax=t1.lmax;
    if(t2.rmax==t2.len)
    tmp.rmax=t2.len+t1.rmax;
    else
    tmp.rmax=t2.rmax;
    tmp.zmax=max(t1.zmax,t2.zmax);
    tmp.zmax=max(tmp.zmax,t1.rmax+t2.lmax);
    return tmp;
}
```